

Theory and Applications of Virtual Reality

Spring 2026

Instructor: Dr. Ibrahim Ghaznavi — TA: Muhammad Qasim

LucidLab AR Science Learning Platform

Team Members

Registration No.	Name
BSCS23070	Muhammad Abdullah
BSCS23118	Abdul Moiz
BSCS23212	Faizan Amir
BSCS23173	Waqas Shoaib
BSCS23176	Sameer

Submission Date: May 10, 2026

1. Introduction (Project Overview)

1.1 Background

The project addresses a practical gap in science education where students often learn theory without immersive experimentation due to lab resource limits, safety concerns, and limited guided feedback outside physical labs. The proposed solution, **LucidLab AR Science Learning Platform**, combines an instructor-facing designer studio with a student-facing mobile AR application. Instructors create experiments and scenes, assign them to classrooms, and evaluate submissions; students run the assigned experiments in AR and receive feedback.

The implemented system is grounded by the repository architecture: a Designer application for experiment and classroom authoring, a backend API service for AI, Firestore, and storage operations, and a Unity AR runtime for student execution. Extended reality is suitable here because it allows interactive object manipulation, scene-based procedural learning, and repeatable low-risk experimentation on mobile devices while preserving instructional structure.

1.2 Problem Statement

Current classroom systems typically separate content authoring, simulation, and assessment into disconnected tools. As a result, instructors struggle to rapidly design interactive experiment logic, while students face limited guided practice and weak continuity between performing an activity and receiving scene-specific feedback.

In this project context, what is missing in current systems is an integrated workflow where:

- instructors can author AR experiment scenes and logic without writing low-level runtime code for each case,
- students can perform assigned AR tasks in a structured classroom-linked flow, and
- submissions and grading can be connected back to the same experiment structure.

1.3 Project Objectives

- Build an instructor Designer studio to create and manage experiments, scenes, and classroom assignments.
- Provide a student mobile AR app that launches assigned experiments and executes scene logic interactively.

- Support marker-based and plane-based AR interaction for flexible classroom deployment.
- Enable visual logic authoring for scene behaviour through a node-based editor.
- Integrate submission and feedback workflows for classroom evaluation.
- Add AI-assisted scene-logic generation (AI Logic Builder) to accelerate authoring using natural language.
- Integrate a multi-modal AI assistant (voice and text) to guide students through experiments and explain concepts.
- Provide pre-built high-fidelity experiments, including Sodium–Chlorine (NaCl) Fusion and Water Synthesis (H₂O), to demonstrate complex AR interactions.

1.4 Implemented Experiments

Two primary high-fidelity demonstrations are integrated to showcase the platform’s capabilities.

Chemical Lab (Flame Colour Test). Set in the `Chemical` scene, this experiment allows students to interact with a Bunsen Burner to observe how different chemicals change the flame’s colour. The scene includes a structured “Exam Mode” where students must identify substances based on their flame emission spectra and submit their answers for evaluation.

Atomic Reaction (Fusion System). A marker-based experiment in the `AtomicReaction` scene where students scan physical element cards. The system renders accurate 3D Bohr models of atoms (Na, Cl, H, O) anchored to the markers. Bringing specific markers together triggers fusion logic in which atoms physically migrate and transform into molecular structures such as Sodium Chloride or Water.

2. Literature Review / Existing Solutions

2.1 Similar Applications

System 1: Labster (VR Science Labs). Labster provides virtual laboratory experiences for science learning with guided tasks and conceptual reinforcement Labster, 2026. It demonstrates the educational value of immersive simulation but is primarily structured as a content platform rather than a classroom-customisable AR scene-authoring environment for instructors.

System 2: HoloLAB Champions (Mixed Reality Chemistry Training). HoloLAB Champions focuses on chemistry lab procedures in mixed reality and emphasises

procedural learning and safety Schell Games, 2026. It validates the usefulness of spatially contextualised learning but is oriented toward packaged training experiences more than instructor-side custom scene-logic composition tied to classroom assignment cycles.

2.2 Gap Analysis

From the implementation evidence in this project, four practical gaps in existing XR systems are addressed.

- **Instructor-side custom logic authoring gap:** many XR applications provide fixed experiences; this platform introduces a Designer workflow with scene-level visual logic editing.
- **Classroom lifecycle integration gap:** this project links design, assignment, student execution, and evaluation in a single pipeline.
- **Hybrid architecture gap:** the system combines web-based authoring and native mobile AR execution, allowing educators to design quickly while students run optimised AR sessions.
- **Real-time guidance gap:** existing systems often lack flexible, hands-free assistance; this project integrates a voice-enabled AI assistant for immediate student support.

The novel features added by this project are **AI-assisted scene-logic generation** integrated into the designer and a **Vapi-powered AI voice assistant** for students, enabling both rapid authoring and immersive guided learning.

3. Target Audience

The target users are:

- **Primary — Instructors/Teachers:** science course educators who need to create and assign interactive AR experiments.
- **Primary — Students:** enrolled learners who complete scene-based AR tasks on mobile devices.
- **Secondary — Teaching Assistants:** staff who may support assignment setup, progress monitoring, and grading workflows.

The audience profile is particularly suited to university-level or advanced school-level science classes where practical experimentation and conceptual visualisation are both required.

4. Use Case & Task Analysis

4.1 Main User Scenario

An instructor logs into the Designer, creates or edits an experiment, adds scenes and object behaviour using the visual logic editor (optionally using the AI Logic Builder for prompt-based generation), and assigns the experiment to a classroom. A student then opens the mobile app, views assigned work, launches AR mode for the experiment, interacts with objects according to scene instructions (assisted by the AI voice chatbot if needed), and submits progress and results for review. The instructor later evaluates submissions and provides feedback.

4.2 Task Flow

1. User launches the application.
2. User selects the classroom and experiment environment.
3. User selects an AR scene from the pre-launch picker.
4. User interacts with 3D objects in the live AR camera view.
5. User receives real-time feedback (scene status, AI hints, logic events).
6. User confirms and submits the experiment result.
7. User exits the system and returns to the classroom dashboard.

5. XR Technology Selection

5.1 Selected Technology

Selected modality: Augmented Reality (AR) with classroom-compatible interaction. AR was selected because the project goal is to integrate experimentation into real physical contexts (classroom or home surfaces) rather than fully isolated virtual environments. The implemented runtime supports marker and plane interaction modes, making AR practical across varying device and environment conditions.

5.2 Hardware Platform

- Android smartphones (ARCore-supported and ARCore-optional metadata configuration).
- iOS devices with ARKit support.

5.3 Software & Tools

- Unity 2022.3.62f3 (LTS) for AR runtime development.
- AR Foundation with ARCore/ARKit providers Unity Technologies, 2026.
- Vuforia Engine for high-precision marker tracking and database management.
- Designer frontend (React/TypeScript) with Rete.js for visual programming (VPL).
- Node.js/Express backend APIs.
- Firebase (authentication and Firestore-backed academic data workflows).
- Supabase storage integration for media and object asset handling.
- Gemini 2.5/3 Flash API for AI scene-logic generation and chatbot intelligence.
- Vapi SDK (@vapi-ai/web) for low-latency AI voice interaction.
- GLTFast for high-performance runtime 3D model loading.
- Unity WebView for persistent hybrid student dashboard integration.

6. System Requirements

6.1 Functional Requirements

1. The system shall allow instructor authentication and protected dashboard access.
2. The system shall allow instructors to create, edit, publish, and delete experiments.
3. The system shall allow scene-level authoring with object creation, transform editing, marker configuration, and logic graph persistence.
4. The system shall allow classroom creation and assignment of published experiments to classroom members.
5. The student application shall launch assigned experiments in AR and load scenes dynamically.
6. The system shall support student submission tracking and instructor evaluation/feedback.
7. The system shall provide AI-assisted generation of scene-logic graph content from natural language prompts.
8. The student app shall include a multi-modal AI assistant supporting both text-based chat and hands-free voice interaction (Vapi).
9. The system shall synchronise object and scene updates between the authoring UI

and Unity runtime for preview and edit workflows.

6.2 Non-Functional Requirements

1. **Usability:** Instructor and student interfaces must support clear workflows with minimal training overhead.
2. **Performance:** Scene and object loading, and editor interactions, should remain responsive under normal classroom-scale content.
3. **Reliability:** Data writes and reads must preserve experiment and submission consistency across sessions.
4. **Compatibility:** The runtime must function on targeted Android and iOS AR-capable devices.
5. **Maintainability:** Architecture should separate authoring UI, backend APIs, and runtime engine responsibilities.

7. Storyboarding

7.1 Storyboard Description

The student journey implemented in the Unity-hosted WebView shell (`student_app.html`) begins at the splash screen, proceeds through authentication, and lands on the home dashboard listing classrooms. The user opens a classroom, reviews assignments, and launches AR through the pre-AR *Select Scene* overlay. In AR, an instruction banner prompts marker scanning before virtual experiment objects are manipulated. The session closes with a submit-confirmation step tied to the assignment workflow before returning to classroom or experiments views.

7.2 Storyboard Frames

The six hand-drawn frames below follow the student UI flow and are aligned with implemented screens in the repository. All frames were produced using the Vincent sketch template (rough pencil/ink, single phone silhouette) as required by course policy.



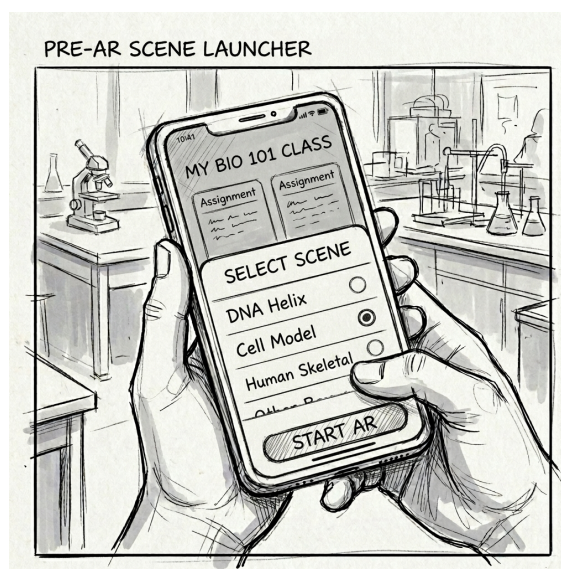
(a) Frame 1: Splash Screen



(b) Frame 2: Login Screen



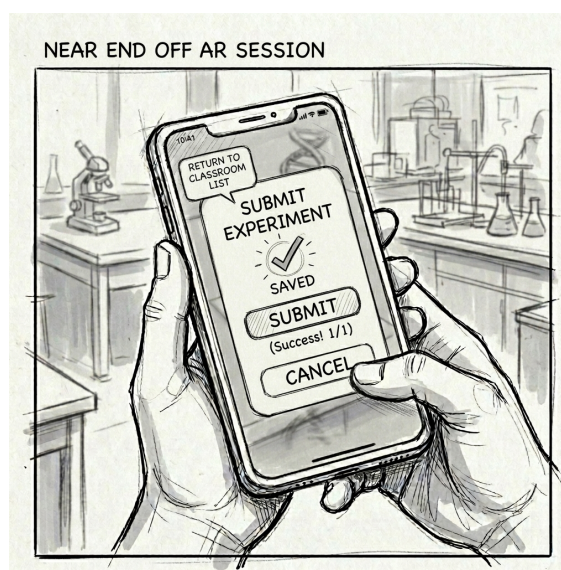
(c) Frame 3: Home / Classrooms



(d) Frame 4: Scene Launcher



(e) Frame 5: Live AR Camera



(f) Frame 6: Submit & Exit

Figure 1: Storyboard — Six-frame student journey (hand-drawn, Vincent sketch template). Frames follow the implemented screen flow: `splash.screen` → `login.screen` → `dashboard` → `ar.scene_picker` → `ar.instruction` → `ar.submit_confirm`.

7.3 Storyboard Analysis

The storyboard supports **immersion** by progressing from branded splash and familiar 2D shell screens into a camera-based AR view with spatial instructions. Each transition increases spatial engagement: the flat 2D dashboard gives way to a real-world camera feed overlaid with guided content, anchoring the experience in the user's physical environment.

Usability is preserved through a strictly linear sequence: authenticate → choose classroom → pick scene → align marker → interact → submit. Each frame introduces only one decision or action, minimising cognitive load and ensuring that first-time users can proceed without on-screen tutorials.

Learning goals are supported because the instruction text and submission step are tied to the same experiment and scene path as in the running application. The AI voice assistant and contextual hint text (visible in Frame 5) reinforce concept understanding during the AR session itself, rather than deferring feedback to a separate post-experiment screen.

8. Development

The development of LucidLab followed an agile, component-oriented approach where architectural decisions were prioritised to allow for the dynamic, AI-assisted workflow. The application is in a fully running state: the student AR application has been built and successfully deployed on Android and iOS mobile devices, while the Designer Studio is hosted as a scalable React web application.

8.1 Team Contributions and Responsibilities

- **Muhammad Abdullah (BSCS23070):** Unity AR mobile app — runtime, WebView shell, Vuforia experiments, and AI (Vapi/Gemini) integration.
- **Abdul Moiz (BSCS23118):** Designer Studio — VPL node editor, scene logic system, and EditorRenderer (Unity WebGL preview).
- **Faizan Amir (BSCS23212):** Designer Studio — React frontend, UI/UX, and component library.
- **Waqas Shoaib (BSCS23173):** Backend — Firebase Auth, Firestore, Gemini AI logic-generation endpoint.
- **Sameer (BSCS23176):** Backend — Supabase storage, asset pipeline, build optimisation.

8.2 AI Systems Implementation & Engineering

Two core AI systems were developed to enhance the platform.

- **Context-Aware VPL Generation:** The backend (`ai.js`) implements a dynamic prompt system that injects real-time scene metadata (object names, types, and existing logic) into Gemini’s system instructions, ensuring that generated logic refers to actual scene assets.
- **Resilient Graph Compilation:** A mechanical sanitisation layer handles common LLM output issues (removing Markdown fences, fixing trailing commas, correcting malformed keys), ensuring 100 % parsing reliability before the graph is sent to the Unity runtime.
- **AI Chatbot via Vapi SDK:** A low-latency voice-enabled AI assistant provides real-time multi-modal guidance by leveraging the platform’s API to understand the student’s current progress and provide context-specific help.

8.3 VPL System Architecture & Node Compilation

The LucidLab Visual Programming Language (VPL) is orchestrated across three distinct layers, bridged by a standardised serialisation format.

- **Designer Studio (Compilation):** Built using the Rete.js framework, a specialised `node_exporter` translates the visual graph into a flat `ExportedNodes` map, separating execution flow from data flow sockets.
- **Editor Renderer (Live Preview):** A Unity-based renderer embedded in the Designer web UI interprets the JSON logic in real-time, allowing instructors to verify 3D interactions before publishing.
- **Mobile Application (Execution):** The Unity AR app features a high-performance interpreter in which the `LogicBuilder` parses nodes into `C# DataInstruction` objects (e.g., `SetPosition`, `ApplyForceOnObject`), executing them in sync with the Unity `Update` loop.

8.4 Bidirectional Editor Synchronisation

To provide a seamless WYSIWYG experience, the platform implements a bidirectional bridge between the React dashboard and the embedded Unity WebGL renderer.

- **React-to-Unity Bridge:** Using `react-unity-webgl` and custom JSLib plugins, the dashboard sends direct messages to Unity to update object properties, focus the camera, or trigger simulations.
- **In-Browser 3D Gizmos:** The renderer integrates `RuntimeTransformHandles`, providing industry-standard Move/Rotate/Scale tools directly within the browser.

- **Real-time Event Feedback:** Unity events (e.g., `unityObjectTransformChanged`) are bubbled back to the React context, ensuring that the database and web UI are updated instantly when an instructor moves an object using the 3D gizmos.

8.5 Simulation Fidelity & Cross-Platform Parity

A key technical achievement is the absolute parity between the Designer’s Live Preview and the Mobile AR App. Both environments share the exact same C# codebase for physics, logic interpretation, and asset handling. A custom `link.xml` configuration preserves critical engine modules (Physics, ARSubsystems, Firebase) in the WebGL build, ensuring that an instructor’s browser preview is a faithful simulation of the student’s AR experience.

8.6 Cloud Orchestration & State Management

- **Unified Data API:** The backend provides a simplified query interface for the Designer Studio, abstracting complex Firestore operations.
- **Role-Based Access Control (RBAC):** Custom middleware enforces strict security by verifying “instructor” or “admin” roles for privileged operations.
- **Centralised Logic Storage:** Experiment logic and scene configurations are stored in Firebase Firestore, enabling instant synchronisation across the Designer and the Student AR App.

8.7 Application Screenshots

The following screenshots demonstrate the LucidLab platform in its fully running state across all major interfaces.

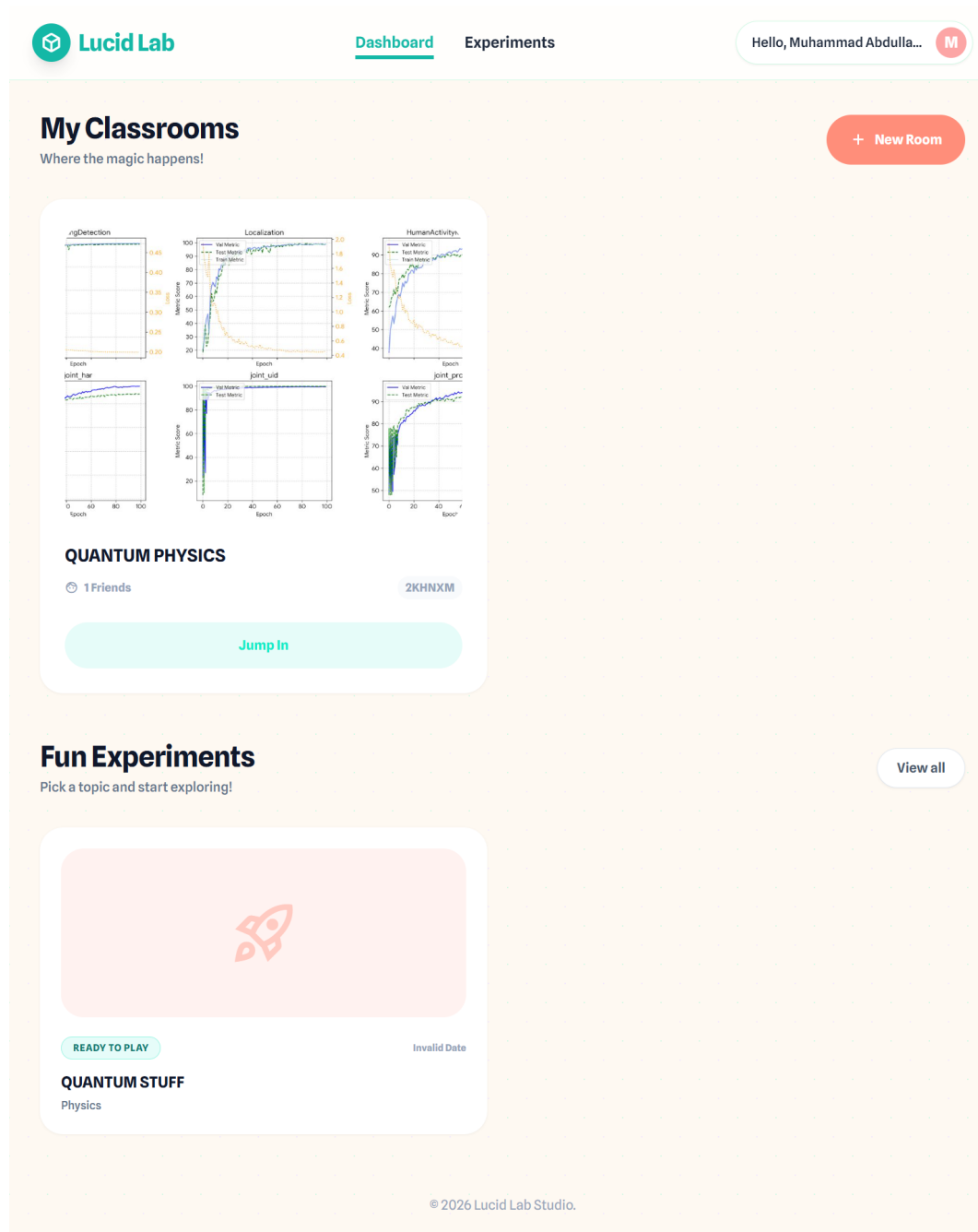


Figure 2: Designer Studio — Classroom management dashboard showing a created “Quantum Physics” classroom with enrolled members and associated experiments.

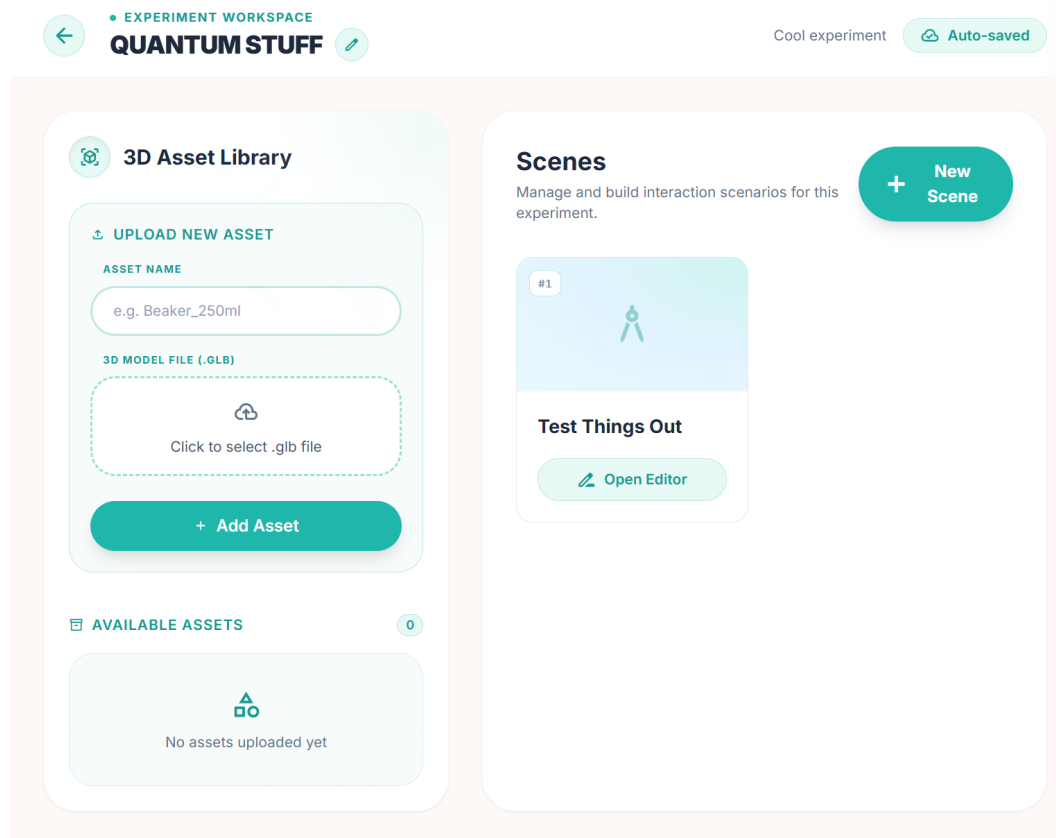


Figure 3: Designer Studio — Experiment workspace with 3D Asset Library (left) and scene management panel (right), showing the *Test Things Out* scene ready for editing.

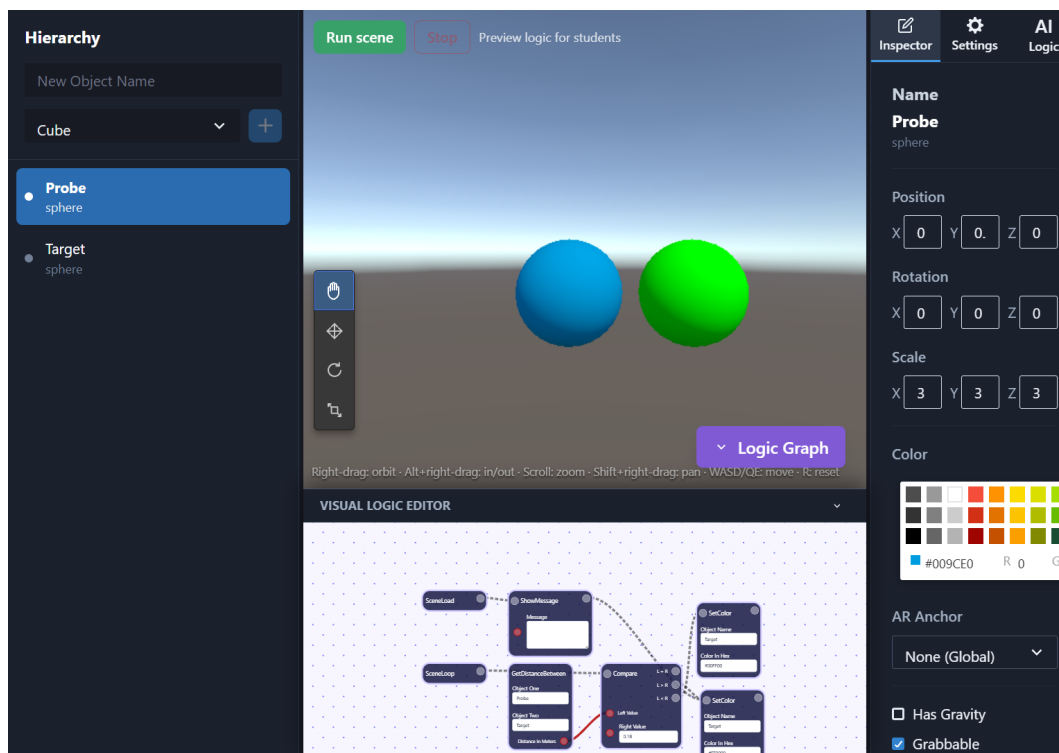


Figure 4: Designer Studio — Scene editor: Unity WebGL viewport (centre) with two spheres in the 3D environment, Hierarchy panel (left), Inspector panel (right), and Visual Logic Editor (bottom) showing connected `GetDistanceBetween` and `SetColor` nodes.



Figure 5: Mobile AR runtime — Chemical Lab (Flame Colour Test) scene deployed on an Android device. Students interact with the Bunsen Burner and select compounds (Ca^{2+} , Ba^{2+} , Sr^{2+} , Cu^{2+} , K^{+} , Na^{+}) to observe corresponding flame colours. Exam Mode is accessible via the *Start Exam* button.

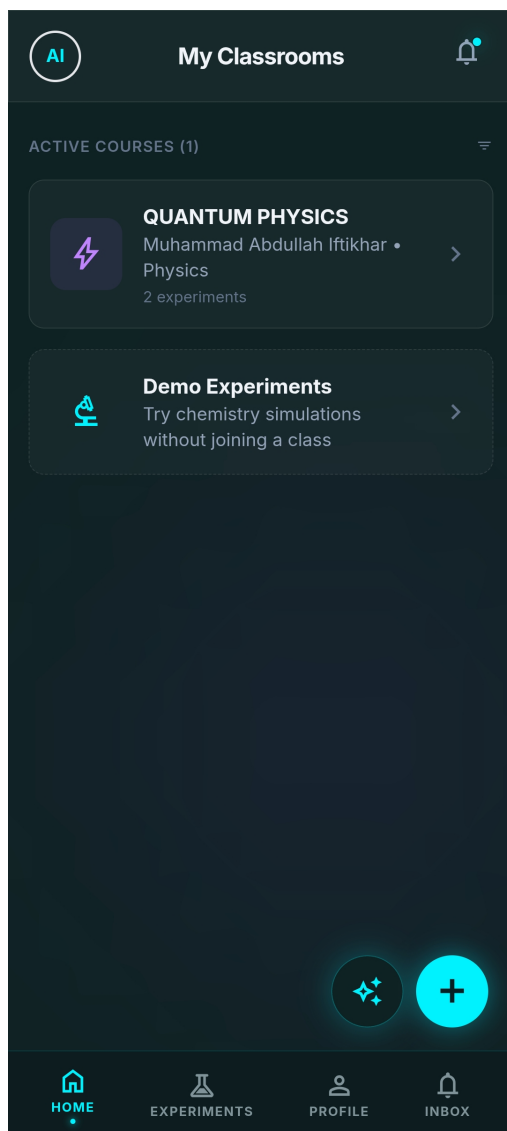


Figure 6: Student mobile app (iOS/Android) — Home screen showing active classrooms (“Quantum Physics”) and a Demo Experiments entry, with bottom navigation bar (Home, Experiments, Profile, Inbox).

8.8 How to Run the Application

8.8.1 *Unity AR App (Student)*

1. Open the LucidLab folder in Unity Hub using Unity 2022.3.62f3 (LTS).
2. Ensure the Universal Render Pipeline (URP) settings are active.
3. In Build Settings, include `LoginScene`, `ARMainScene`, `Chemical`, and `AtomicReaction`.
4. Build and run on a supported ARCore (Android) or ARKit (iOS) device.
5. Use the included `VuforiaTargets` images (located in `Assets/Textures/VuforiaTargets`) to trigger the experiments.

8.8.2 Designer Studio (Instructor)

1. Navigate to the `Designer` directory.
2. Run `npm install` followed by `npm run dev`.
3. Access the dashboard at `localhost:3000` to create and manage classrooms.

9. Limitations & Risks

- **Technical limitations:**

- Hybrid architecture complexity (web authoring + backend API + Unity runtime) increases integration overhead.
- Documentation drift exists between some design documents and current implemented backend paths and endpoints.
- AI generation for logic graphs can occasionally require prompt refinement for highly complex conditional branches.

- **Hardware constraints:**

- AR quality depends on device sensors, camera, and platform AR support.
- Marker tracking quality can degrade in poor lighting or on low-feature surfaces.
- Voice AI performance is sensitive to network latency and ambient classroom noise.

- **Skill limitations:**

- Advanced AR interaction tuning and robust cross-platform WebView-media behaviour require specialised expertise.
- Tuning LLM system prompts for consistent VPL graph generation is an ongoing iterative process.

- **Time constraints:**

- Given semester timelines, advanced fluid mechanics and high-fidelity simulation features were constrained; focus remained on a robust educational workflow and interaction pipeline including AI integration.

Implementation-vs-Documentation Drift Note

During repository evidence review, several documented architectural claims (e.g., some cloud-functions and AI endpoint descriptions) did not fully match current implemented backend routes. This report prioritises implemented code evidence for accuracy and treats such mismatches as a maintainability risk rather than a product intent failure.

10. Conclusion

LucidLab AR Science Learning Platform presents a practical, evidence-backed approach for immersive classroom science learning by combining instructor-side authoring, student-side mobile AR execution, and classroom evaluation workflows. The project demonstrates that AR can be used not only for visualisation but also for structured interactive learning tasks linked to instructional outcomes. The implemented architecture — spanning a visual programming language, real-time cross-platform preview, AI-assisted authoring, and a voice-guided student experience — establishes a strong foundation for future enhancements such as richer simulation behaviour, stronger test automation, and tighter production-hardening of backend and media pipelines.

11. References

References

- Azuma, R. T. (1997). A survey of augmented reality. *Presence: Teleoperators and Virtual Environments*, 6(4), 355–385. <https://doi.org/10.1162/pres.1997.6.4.355>
- Billinghurst, M., Clark, A., & Lee, G. (2015). A survey of augmented reality. *Foundations and Trends in Human-Computer Interaction*, 8(2–3), 73–272. <https://doi.org/10.1561/11000000049>
- Labster. (2026). Virtual labs for science education. Retrieved May 1, 2026, from <https://www.labster.com>
- LucidLab Team. (2026). LucidLab project repository and implementation artefacts [Internal course project source, Spring 2026].
- Schell Games. (2026). HoloLAB Champions. Retrieved May 1, 2026, from <https://www.hololabchampions.com>
- Unity Technologies. (2026). AR Foundation documentation. Retrieved May 1, 2026, from <https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@latest>